Boa noite! Ótimo feedback — é cirúrgico e dá para resolver bem. Deixa eu traduzir o que eles querem:

Problema 1 — `days_in_pipeline` contribui 0 pts para todos os deals O dataset é de 2016-2017, então todos os deals abertos têm 3.000+ dias e caem sempre na faixa "180+ dias = 0 pts". A feature existe mas não diferencia ninguém — é ruído. Eles querem que você a substitua por algo que realmente discrimine os deals.

Problema 2 — Process log superficial Os erros documentados foram só de UX (cor, decimal). Eles querem ver raciocínio analítico: quais hipóteses você testou, por que escolheu essas 6 features e não outras, o que descartou e por quê.

Antes de partir para as correções, me responde:

Para o problema 1, temos algumas opções para substituir `days_in_pipeline`:

1. **Velocidade de progressão** — quantos dias o deal levou de Prospecting para Engaging (deals que avançaram rápido têm mais chance de fechar)
2. **Proximidade do close_date** — quantos dias faltam para a data prevista de fechamento
3. **Sazonalidade** — mês do engage_date (alguns meses convertem mais)

Qual faz mais sentido para você? Ou quer que eu analise qual discrimina melhor com os dados reais antes de decidir?

achei interesante o de sazonalidade, queria entender melhor como funcionaria o 1 e o 2, talvez tenham potencial

18 de abr.

Boa análise. Deixa eu explicar os três:

Opção 1 — Velocidade de progressão

Mede quantos dias o deal ficou em Prospecting antes de virar Engaging. Um deal que avançou em 7 dias é mais quente que um que ficou parado 90 dias antes de engajar.

Problema: só funciona para deals que já estão em Engaging. Os 500 deals em Prospecting não teriam esse dado ainda — receberiam um valor neutro.

Opção 2 — Proximidade do close_date

Mede quantos dias faltam para a data prevista de fechamento. Deal com close_date em 15 dias merece mais atenção que um com close_date em 6 meses.

Problema: precisa verificar nos dados se `close_date` está preenchido nos deals abertos — se estiver vazio como o `close_value`, a feature não funciona.

Opção 3 — Sazonalidade

Mede o mês em que o deal entrou no pipeline. Baseia-se no win rate histórico por mês — se deals que começaram em março convertem mais, deals atuais que começaram em março ganham mais pontos.

Vantagem: usa o histórico de Won/Lost que já temos, igual ao win rate por setor e vendedor. É consistente com a lógica do modelo.

Antes de decidir, vale verificar nos dados se `close_date` existe nos deals abertos. Manda isso para o Claude Code:

```
Analise os dados em data/ e me responda:

1. Nos deals com stage Engaging ou Prospecting, quantos têm
   close_date preenchido? Mostre exemplos.

2. Para os deals históricos (Won e Lost), qual é a distribuição
   de win rate por mês do engage_date? Algum mês se destaca?

3. Para deals em Engaging, existe a data de quando saíram de
   Prospecting? Ou só temos engage_date como referência?
```

Com esse diagnóstico a gente escolhe a feature com mais potencial real nos dados.

boa, vou manda para ele... pensei aqui também que ao inves de deixar a ia como opcional, colocar ela ja imbutida, para ela fazer analise, ou ate mesmo colocarmos um chat, para os vendedores pedirem analise especifficas a ia, com metricas especificas

18 de abr.

Ótimas ideias — e ambas elevam muito o nível da entrega. Deixa eu avaliar cada uma:

Ideia 1 — IA embutida (análise automática sem precisar clicar)

Em vez do botão "Analisar com IA", a recomendação já aparece carregada para os top 10 deals Tier A. O vendedor abre o app e já vê a ação recomendada sem precisar fazer nada.

Vantagem: experiência muito mais próxima de um produto real. Desvantagem: consome API para todos os deals ao carregar — pode ficar lento e caro em produção. Solução: cachear as recomendações por 24h.

Ideia 2 — Chat com a IA sobre o pipeline

Um campo de chat onde o vendedor digita perguntas como:

- "Quais são meus melhores deals do setor de tecnologia?"
- "Por que o deal da Betasoloin tem score 71?"
- "Quais deals devo priorizar essa semana?"

A IA recebe a pergunta + os dados do pipeline filtrado do vendedor e responde em linguagem natural.

Vantagem: diferencial enorme — nenhuma solução de scoring básica tem isso. É exatamente o tipo de coisa que impressiona como "AI Master". Desvantagem: mais complexo de implementar, mas totalmente viável com Streamlit + Claude API.

Minha recomendação: faça as duas, nessa ordem:

1. Primeiro resolve o problema do scoring (feature ruim) — isso é bloqueante para aprovação
2. Depois adiciona a IA embutida nos top 10 — melhoria rápida
3. Por último o chat — diferencial que te destaca

Manda o diagnóstico dos dados para o Claude Code primeiro. Enquanto ele responde a gente planeja o chat.

a ideia do chat só, acho que seria muito bom

18 de abr.

Concordo — é o diferencial mais forte. Um vendedor digitando "quais meus melhores deals essa semana?" e recebendo uma resposta contextualizada com os dados reais do pipeline dele é algo que nenhum scoring básico entrega.

Mas vamos na ordem certa para não perder a aprovação:

Passo 1 agora: aguarda o diagnóstico dos dados que o Claude Code vai retornar sobre o `close_date` e sazonalidade — isso resolve o problema bloqueante do scoring.

Passo 2 depois: implementamos o chat.

O Claude Code já retornou alguma coisa?

aqui vai a resposta do diagnostico:

Respostas 1. Deals abertos com `close_date` preenchido Zero. Nenhum dos 2.089 deals abertos (Engaging + Prospecting) tem `close_date`. Faz sentido — a data só é preenchida quando o deal fecha (Won ou Lost). Não há exceção. 2. Win rate por mês do `engage_date` PeríodoWin RateVolumeOut/2016100%7 deals (amostra pequena)Nov–Dez/2016~81%102–196 dealsJan/201773%247 dealsFev–Mar/201766–67%420–655 dealsJul/201755,4%796 deals (maior volume)Dez/201777,7%103 deals Padrão claro: win rate cai de ~81% no início (out–dez/2016) para ~55–57% no meio do ciclo (jul e out/2017), com leve recuperação no final. Isso sugere que os primeiros deals eram mais qualificados ou os vendedores ainda estavam numa fase inicial mais seletiva. Jul/2017 é o pior mês com volume representativo — maior concorrência ou deals mais difíceis entrando no pipeline. 3. Data de transição Prospecting → Engaging Não existe. O dataset tem apenas 8 colunas, e a única data de entrada de estágio é `engage_date` — definida no metadata como "data em que o stage Engaging foi iniciado". Não há coluna de quando o deal entrou em Prospecting nem de quanto tempo ficou lá antes de avançar. Para deals ainda em Prospecting, `engage_date` está completamente nula (todos os 500 deals). Ou seja: só sabemos quando virou Engaging, nunca quando começou como Prospecting.

Mostrar mais
18 de abr.

Diagnóstico claro. A decisão é fácil agora:

Sazonalidade é a única opção viável — `close_date` não existe nos deals abertos, e velocidade de progressão não funciona para os 500 deals em Prospecting.

E o padrão é real e interessante — win rate cai de 81% no início para 55% no pico de volume. Isso discrimina deals de verdade.

A lógica seria: pega o mês do `engage_date` de cada deal aberto, busca o win rate histórico daquele mês, converte em pontos. Deals que entraram em meses historicamente melhores ganham mais pontos.

Problema: os deals abertos são de 2016-2017 também, então o mês do engage já é fixo — não muda. Mas ainda discrimina porque deals que engajaram em outubro vs julho têm win rates históricos diferentes.

Manda isso para o Claude Code:

```
Substitua a feature days_in_pipeline pela feature sazonalidade
no scoring.py.

A nova feature funciona assim:

1. Calcule o win rate histórico por mês de engage_date usando
   os deals Won e Lost. Retorne um dict {mes: win_rate} onde
   mes é o número do mês (1-12).

2. Na função score_deal, substitua a feature days_in_pipeline
   (0-20 pts) pela feature sazonalidade (0-20 pts):
   - Pegue o mês do engage_date do deal
   - Busque o win rate histórico daquele mês
   - round(month_wr.get(mes, 0.5) * 20)

3. Atualize compute_win_rates() para retornar também o dict
   de win rates por mês.

4. Atualize score_all() para passar o novo dict para score_deal.

5. Atualize o label no breakdown de "Tempo no pipeline"
   para "Sazonalidade (mês)".

6. No app.py, atualize o caption da Aba 1 explicando a nova
   feature: "Sazonalidade: win rate histórico do mês de entrada
   no pipeline"

Após alterar, mostre:
- A distribuição de pontos da nova feature nos deals abertos
- Se ela realmente discrimina os deals (valores diferentes)
- Teste com: streamlit run app.py --server.headless true
```

Me mostra o retorno quando terminar.

Leu um arquivo, pesquisado código Agora faço todas as edições: Editado 4 arquivos Agora valido e analiso a distribuição da nova feature: Executado 2 comandos Feito. Resultados: Distribuição da nova feature MêsWin rate históricoPontosDez80,3%16 ptsJan73,3%15 ptsNov69,4%14 ptsFev/Mar/Jun/Ago/Set62–67%13 ptsMai62,2%12 ptsAbr/Jul/Out55–57%11 ptsProspecting s/ datafallback 50%10 pts 7 valores distintos (contra 1 único valor que

`days_in_pipeline` produzia — sempre 0). Impacto no score geral MétricaAntesDepoisTier A7 deals31 dealsScore mín/máx44–7139–87Desvio padrão4,08,4 A feature agora discrimina de verdade: desvio padrão dobrou de 4 para 8,4 pts, e o range passou de 27 para 48 pontos. Deals que entraram em Dezembro/Janeiro têm vantagem histórica real de ~5 pts sobre os de Julho.

Mostrar mais
18 de abr.

Excelente resultado. A feature agora discrimina de verdade — desvio padrão dobrou e o Tier A passou de 7 para 31 deals, que é muito mais útil para o vendedor.

Agora vamos implementar o chat. Manda isso para o Claude Code:

Adicione uma nova aba "Chat com IA" no app.py, entre as abas "Análise IA" e "Gestor".

O chat funciona assim:

1. O vendedor seleciona o próprio nome no filtro de Vendedor na sidebar — o chat usa apenas os deals visíveis na view filtrada atual.

2. Interface de chat com histórico de mensagens usando st.session_state para manter a conversa.

3. Campo de input na parte inferior: st.chat_input("Pergunte sobre seu pipeline...")

4. Quando o vendedor envia uma mensagem, a IA recebe:
   - Contexto completo: os dados dos deals visíveis em formato resumido (account, stage, score, tier, valor, setor, win rates)
   - Histórico da conversa atual
   - A pergunta do vendedor

5. System prompt para a IA:
   "Você é um assistente de vendas especialista em análise de pipeline. Responda sempre de forma direta e acionável. Use os dados reais do pipeline fornecidos. Cite números e deals específicos nas respostas. Nunca invente dados que não estejam no contexto."

6. Exemplos de perguntas sugeridas (mostrar como botões

clicáveis acima do input quando o chat estiver vazio):
- "Quais meus 3 melhores deals para fechar essa semana?"
- "Por que meu score médio está baixo?"
- "Quais setores têm mais chance de fechar?"
- "Quais deals devo abandonar?"

7. Se não tiver API key: mostrar st.warning pedindo a chave, igual às outras abas.

8. Limitar o contexto enviado para a IA a no máximo 50 deals (os top 50 por score) para não estourar tokens.

9. Adicione um botão "Limpar conversa" que reseta o st.session_state do histórico.

Após implementar teste com:
streamlit run app.py --server.headless true
Me mostra o retorno quando terminar.

acredito que na API Key ao inves de limitarmos só na antropic, podemos deixar aberto para uma IA de preferencia do usuario

18 de abr.

Boa ideia conceitualmente, mas para este challenge eu recomendo manter só Claude por dois motivos:

1. **Tempo** — implementar multi-provider (Claude + OpenAI + Gemini) adiciona complexidade que pode introduzir bugs antes do prazo
2. **Contexto do challenge** — você já está usando Claude Code e Claude.ai como ferramentas. Manter Claude na app é consistente com o process log e mostra domínio de uma stack, não dispersão

O que você pode fazer de forma simples é deixar o **modelo** configurável dentro da Anthropic — por exemplo, deixar o usuário escolher entre `claude-sonnet-4-20250514` e `claude-haiku` (mais barato e rápido). Isso mostra consciência de custo sem adicionar complexidade.

Quer adicionar isso ou segue com o chat primeiro?

isso, adicione isso

18 de abr.

Manda isso para o Claude Code:

Faça dois ajustes no app.py:

1. Na sidebar, abaixo do campo de API key, adicione um
   selectbox para o usuário escolher o modelo:

```
modelo = st.selectbox(
    "Modelo Claude",
    options=[
        "claude-sonnet-4-20250514",
        "claude-haiku-4-5-20251001"
    ],
    index=0,
    help="Sonnet: mais preciso. Haiku: mais rápido e barato."
)
```

2. Passe o modelo selecionado para todas as chamadas da
   Claude API no app — tanto na Aba Análise IA quanto
   no Chat (quando implementado).

3. Atualize a função get_recommendation() em ai_insights.py
   para aceitar um parâmetro model="claude-sonnet-4-20250514"
   e use ele na chamada da API.

Após alterar teste com:
streamlit run app.py --server.headless true
Quando retornar implementamos o chat na sequência.

Agora o sidebar e as chamadas no app.py: Editado 3 arquivos, executado um comando Feito. Três mudanças cirúrgicas:

- `ai_insights.py`: `get_recommendation()` e `chat_completion()` agora aceitam `model=` com default `claude-sonnet-4-20250514` — retrocompatível

- `app.py` sidebar: `st.selectbox("Modelo Claude", ...)` posicionado logo abaixo do campo de API key
- `app.py` chamadas: ambas as chamadas (Análise IA e Chat) passam `model` para as funções

18 de abr.

Perfeito. Agora implementa o chat. Manda isso para o Claude Code:

```
Adicione uma nova aba "Chat com IA" no app.py, entre as abas
"Análise IA" e "Gestor".

O chat funciona assim:

1. Usa os deals visíveis na view filtrada atual — se o vendedor
   filtrou pelo próprio nome na sidebar, o chat responde só
   sobre os deals dele.

2. Histórico de mensagens mantido via st.session_state.

3. Campo de input: st.chat_input("Pergunte sobre seu pipeline...")

4. Quando o vendedor envia uma mensagem, monta o payload para
   a Claude API com:
   - System prompt:
     "Você é um assistente de vendas especialista em análise de
     pipeline B2B. Responda sempre de forma direta e acionável.
     Use apenas os dados reais do pipeline fornecidos no contexto.
     Cite contas e números específicos nas respostas.
     Nunca invente dados que não estejam no contexto."
   - Contexto dos dados: top 50 deals por score da view filtrada,
     incluindo account, deal_stage, score, tier, sector,
     sales_agent, effective_value, win_rate do setor e vendedor
   - Histórico completo da conversa
   - Mensagem do vendedor

5. Quando o chat estiver vazio, mostre 4 botões de sugestão
   clicáveis que preenchem o input:
   - "Quais meus 3 melhores deals para fechar essa semana?"
   - "Quais setores têm mais chance de fechar?"
   - "Por que meu score médio está baixo?"
   - "Quais deals devo abandonar?"

6. Ao clicar num botão de sugestão, dispara a pergunta
   automaticamente via st.session_state.
```

7. Se não tiver API key: st.warning pedindo a chave.

8. Botão "Limpar conversa" que reseta o histórico.

9. Use st.chat_message("user") e st.chat_message("assistant")
   para renderizar as mensagens.

10. Passe o model selecionado na sidebar para a chamada da API.

Após implementar teste com:
streamlit run app.py --server.headless true
Me mostra o retorno quando terminar.

corrija esses erros, no nome dos modelos do claude está assim = claude-sonnet-4-20250514. Deixe o nome mais limpo dessa forma 'claude-sonnet-4'.

na aba Chat com Ia, deixe um campo para o usuario escrever suas perguntas, e que ele consiga colocar filtros ali mesmo, como pelo nome do vendedor.

18 de abr.

Manda isso para o Claude Code:

Faça três ajustes no app.py e ai_insights.py:

1. NOMES DOS MODELOS — deixe os labels mais limpos no selectbox
   da sidebar, mas mantenha o model ID correto na chamada da API:

   ```
   options = {
       "claude-sonnet-4": "claude-sonnet-4-20250514",
       "claude-haiku-4": "claude-haiku-4-5-20251001"
   }

   label_selecionado = st.selectbox("Modelo Claude", list(options.keys()))
   model = options[label_selecionado]
   ```

2. CHAT — campo de input e filtros inline na própria aba:

   Acima do histórico de mensagens, adicione uma linha com
   3 colunas:
   - Col 1: selectbox "Vendedor" (opção "Todos" + lista de

vendedores dos deals) — filtra o contexto enviado para a IA
- Col 2: selectbox "Tier" (Todos / A / B / C)
- Col 3: selectbox "Região" (Todos + lista)

Esses filtros são independentes dos filtros da sidebar —
afetam apenas o contexto do chat, não a tabela Pipeline.

O contexto enviado para a IA usa os filtros do chat,
não os da sidebar.

3. CHAT — campo de escrita:

Mantenha o st.chat_input na parte inferior da aba.
Os botões de sugestão aparecem só quando o histórico
está vazio.

Após alterar teste com:
streamlit run app.py --server.headless true
Me mostra o retorno quando terminar.

na aba pipeline, acho que é desnecessario mostrar na tabela os dias no pipeline, o que voce acha? queria saber tambem aonde posso ver na aplicação a parte onde vemos a analise de sazonalidade

18 de abr.

Sobre remover dias no pipeline da tabela: concordo totalmente. A coluna não agrega valor para o vendedor — ele não precisa saber que o deal tem 3.200 dias, isso é ruído. A tabela fica mais limpa sem ela.

Sobre a sazonalidade: atualmente ela aparece só no gráfico de breakdown da Aba Análise IA, quando você seleciona um deal específico — aparece como "Sazonalidade (mês): X pts". Não tem uma visão agregada mostrando, por exemplo, quais meses têm mais deals ou como a sazonalidade afeta o score geral.

Se quiser, posso sugerir adicionar na Aba Gestor um gráfico mostrando o win rate histórico por mês — seria uma visualização útil para o manager entender em quais períodos o time converte melhor.

Manda os dois ajustes para o Claude Code:

Dois ajustes no app.py:

1. TABELA PIPELINE — remova a coluna days_in_pipeline
   da tabela na Aba 1. Mantenha todas as outras colunas.

2. ABA GESTOR — adicione um novo gráfico após o scatter plot:
   Título: "Win rate histórico por mês de entrada no pipeline"
   - Barras verticais com o mês no eixo X e win rate (%) no eixo Y
   - Cor verde para meses acima de 65%, amarelo entre 55-65%,
     vermelho abaixo de 55%
   - Mostre o volume de deals de cada mês como texto acima
     de cada barra
   - Use os dados históricos de Won e Lost para calcular

Teste com: streamlit run app.py --server.headless true

remova o campo dias no pipeline de todas as abas, e na parte de sazonalidade, deixe claro sobre qual mes, com nome e etc... quero passar algo que seja claro de ler e interpretar

18 de abr.

Manda isso para o Claude Code:

Dois ajustes no app.py e scoring.py:

1. REMOVER days_in_pipeline de todo o app:
   - Remova da tabela da Aba 1 Pipeline
   - Remova da tabela de detalhes do deal na Aba 2 Análise IA
   - Remova do contexto enviado para a IA no Chat
   - Remova do scatter plot da Aba Gestor (use score no eixo Y
     e effective_value no eixo X como substituto)

2. SAZONALIDADE — deixe legível em todo o app:

   No breakdown do gráfico da Aba 2, em vez de mostrar
   "Sazonalidade (mês): 13 pts", mostre o nome completo
   do mês em português + win rate histórico:
   "Sazonalidade — Julho (55% win rate): 11 pts"

   No gráfico da Aba Gestor "Win rate histórico por mês":
   - Eixo X: nome completo do mês em português (Janeiro,
     Fevereiro, etc.) ordenado cronologicamente

- Eixo Y: win rate em porcentagem (ex: 73%)
- Texto acima de cada barra: "73% (247 deals)"
- Cores: verde=#4ade80 se wr>=65%, amarelo=#fbbf24 se wr>=55%, vermelho=#f87171 se wr<55%
- Título: "Win rate histórico por mês de entrada no pipeline"
- Subtítulo em caption: "Base: deals Won e Lost históricos. Meses com win rate alto geram mais pontos de sazonalidade no score."

No contexto do chat, quando mencionar sazonalidade use: "Sazonalidade: entrou em Julho (win rate histórico: 55%)"

Teste com: streamlit run app.py --server.headless true
Me mostra o retorno quando terminar.

queria deixar o front end com o desing do g4, podemos pegar como referencia a pagina de vagas deles, lá da pra ter uma referencia do design e tudo mais

18 de abr.

Boa ideia — personalizar o visual para o estilo do G4 mostra atenção ao detalhe e deixa a entrega mais profissional.

Antes de montar o prompt, preciso ver o site deles para pegar as referências certas de cores, tipografia e estilo. Deixa eu acessar:

Failed to fetch: https://g4educacao.com/vagas

Failed to fetch: https://g4educacao.com/vagas

Não consegui acessar diretamente. Me manda o link exato da página de vagas deles que você quer usar como referência — ou tira um print e cola aqui que eu analiso as cores, fontes e estilo para montar o prompt certinho.

https://ats.g4business.com/careers

18 de abr.

Fetched: G4 Talent Flow - Sistema de Gestão de Talentos

Fetched: G4 Talent Flow - Sistema de Gestão de Talentos

O site não renderizou o conteúdo via fetch. Me manda um print da página que eu analiso as cores e estilo direto da imagem.

18 de abr.

Perfeito. Consegui extrair o design system do G4:

- **Fundo escuro:** `#0a1628` (azul marinho profundo)
- **Cor dourada:** `#c9a84c` (logo, ícones, destaques)
- **Texto primário:** branco `#ffffff`
- **Texto secundário:** cinza claro `#94a3b8`
- **Cards:** `#0f2133` com borda sutil
- **Botão primário:** `#0f172a` fundo escuro com texto branco
- **Estilo geral:** clean, dark, sofisticado — sem cores vibrantes exceto o dourado

Manda isso para o Claude Code:

Aplique o design system do G4 Educação no app.py via st.markdown com CSS customizado. O estilo deve ser inspirado no site ats.g4business.com/careers.

Paleta de cores:
- Fundo principal: #0a1628
- Fundo cards/sidebar: #0f2133
- Cor de destaque dourada: #c9a84c
- Texto primário: #ffffff
- Texto secundário: #94a3b8
- Bordas: #1e3a5f
- Botões: fundo #c9a84c, texto #0a1628

Aplique via st.markdown(unsafe_allow_html=True) no início do app, antes de qualquer componente:

1. FUNDO GERAL:
   - Cor de fundo da página: #0a1628
   - Cor de fundo da sidebar: #0f2133
   - Texto geral: #ffffff

2. HEADER DO APP:
   - Substitua o título "Lead Scorer — Priorização de Pipeline"
     por um header customizado em HTML:
     - Logo texto "G4" na cor dourada #c9a84c, bold, 28px
     - Ao lado: "Lead Scorer" em branco, 24px
     - Subtítulo abaixo: "Priorização inteligente de pipeline"
       em #94a3b8, 14px
   - Renderize com st.markdown(unsafe_allow_html=True)

3. MÉTRICAS:
   - Cards de métrica com fundo #0f2133, borda 1px #1e3a5f
   - Valor da métrica na cor dourada #c9a84c, 28px, bold
   - Label em #94a3b8, 12px uppercase

4. ABAS:
   - Aba ativa com underline dourado #c9a84c
   - Texto das abas em branco

5. TABELA PIPELINE:
   - Cabeçalho: fundo #0f2133, texto #c9a84c, uppercase, 12px
   - Linhas alternadas: #0a1628 e #0d1e35
   - Tier A: fundo #0f2a1a, texto #4ade80
   - Tier B: fundo #2a1f0a, texto #c9a84c
   - Tier C: fundo #2a0f0f, texto #f87171
   - Borda entre linhas: #1e3a5f

6. SIDEBAR:
   - Labels dos filtros em #c9a84c, 11px uppercase
   - Selectboxes com fundo #0a1628, borda #1e3a5f
   - Botão "Atualizar scores": fundo #c9a84c, texto #0a1628,
     bold, border-radius 6px

7. BOTÕES DE SUGESTÃO DO CHAT:
   - Fundo #0f2133, borda #c9a84c, texto #c9a84c
   - Hover: fundo #c9a84c, texto #0a1628

8. GRÁFICOS PLOTLY:
   - Atualize todos os gráficos para usar:
     paper_bgcolor="#0a1628"
     plot_bgcolor="#0f2133"
     font=dict(color="#ffffff")
     gridcolor="#1e3a5f"
   - Barras Tier A: #4ade80
   - Barras Tier B: #c9a84c
   - Barras Tier C: #f87171

9. CAPTION E RODAPÉ:
   - Texto em #94a3b8, 12px

Após aplicar teste com:
streamlit run app.py --server.headless true

Me mostre quais elementos foram alterados e se houve
alguma limitação do Streamlit que impediu algum estilo.
Me mostra o retorno quando terminar.

depois que foi mudado o design apareceu esse erro em todas as abas:

18 de abr.

Manda isso para o Claude Code:

Corrija o erro: TypeError: update_layout() got multiple values
for keyword argument 'margin'

O problema é que o dicionário G4_PLOTLY está sendo passado com
**G4_PLOTLY dentro de update_layout() que já tem margin definido
explicitamente em algum gráfico.

Corrija assim:
1. Remova o parâmetro margin do dicionário G4_PLOTLY
2. Aplique margin separadamente em cada gráfico que precisar:
   fig.update_layout(**G4_PLOTLY)
   fig.update_layout(margin=dict(l=20, r=20, t=40, b=20))

Verifique todos os gráficos do app.py e corrija onde houver
conflito de parâmetros duplicados no update_layout().

Teste com: streamlit run app.py --server.headless true

quero deixar o nome neste padrao: G4Lead Scorer

18 de abr.

Manda isso para o Claude Code:

No header do app, atualize o título para seguir este padrão visual:

G4Lead Scorer

Sem espaço entre G4 e Lead Scorer, sem separadores, sem subtítulo.
Aplique via st.markdown(unsafe_allow_html=True).

so fazer um pequena correção, a aba deploy que esta em um cinza escuro esta tampando o nome da pagina no inicio, e no menu ao lado esquerdo que deixar o nome com o mesmo padrao do G4Lead Scorer

18 de abr.

Manda isso para o Claude Code:

Duas correções visuais no app.py:

1. BOTÃO DEPLOY — está sobrepondo o header. Esconda o botão
   de deploy do Streamlit via CSS:

```
st.markdown("""
<style>
.stDeployButton { display: none !important; }
header[data-testid="stHeader"] { background: transparent !important; }
</style>
""", unsafe_allow_html=True)
```

2. SIDEBAR — atualize o título/nome exibido na sidebar para
   seguir o mesmo padrão visual do header:

```
st.sidebar.markdown("""
<div style="margin-bottom: 24px;">
  <span style="color:#c9a84c; font-weight:800; font-size:22px;">G4</span><span
style="color:#ffffff; font-weight:400; font-size:22px;">Lead Scorer</span>
</div>
""", unsafe_allow_html=True)

Remova qualquer st.sidebar.title() ou st.title() que
estiver duplicando o nome.

Teste com: streamlit run app.py --server.headless true
```

corrigir esse input, ele esta com um retangulo de outra cor no fund do texto

18 de abr.

Manda isso para o Claude Code:

```
Corrija o estilo do st.chat_input no app.py. O campo está
com fundo inconsistente — aparece um retângulo de cor
diferente atrás do texto placeholder.

Adicione este CSS no bloco de estilos globais:

div[data-testid="stChatInput"] {
  background-color: #0f2133 !important;
  border: 1px solid #1e3a5f !important;
  border-radius: 8px !important;
}

div[data-testid="stChatInput"] textarea {
  background-color: #0f2133 !important;
  color: #ffffff !important;
}

div[data-testid="stChatInput"] textarea::placeholder {
  color: #94a3b8 !important;
}
```

```
div[data-testid="stChatInputContainer"] {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
}

Teste com: streamlit run app.py --server.headless true
```

quero deixar o input todo com uma cor só, um background só

18 de abr.

Manda isso para o Claude Code:

```
Corrija o campo de chat input para ter um background único
e consistente, sem retângulos internos de cores diferentes.

Substitua todos os estilos do chat input por este bloco único:

section[data-testid="stBottom"] {
    background-color: #0a1628 !important;
}

section[data-testid="stBottom"] > div {
    background-color: #0a1628 !important;
}

div[data-testid="stChatInputContainer"],
div[data-testid="stChatInputContainer"] > div,
div[data-testid="stChatInputContainer"] textarea {
    background-color: #0f2133 !important;
    color: #ffffff !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
    box-shadow: none !important;
}

div[data-testid="stChatInputContainer"] textarea::placeholder {
    color: #94a3b8 !important;
}

Remova qualquer outro estilo CSS relacionado ao chat input
```

que esteja no código para evitar conflito.

Teste com: streamlit run app.py --server.headless true

o light mode ficou todo estranho, devemos corrigir para o design ficar padrao do g4

18 de abr.

Está quase lá — o header e sidebar ficaram ótimos. Os problemas são:

1. **Botões de sugestão** estão brancos (light mode)
2. **Botão limpar conversa** está branco
3. **Chat input** ainda com fundo claro
4. **Botão "Atualizar scores"** ficou vermelho — deveria ser dourado

Manda isso para o Claude Code:

Corrija os elementos que ficaram com fundo branco/claro no app.py.
O app deve ser 100% dark com a paleta G4. Adicione estes estilos
no bloco CSS global:

```
/* Botões gerais */
.stButton > button {
    background-color: #0f2133 !important;
    color: #c9a84c !important;
    border: 1px solid #c9a84c !important;
    border-radius: 6px !important;
}

.stButton > button:hover {
    background-color: #c9a84c !important;
    color: #0a1628 !important;
}

/* Botão Atualizar scores — dourado sólido */
div[data-testid="stSidebar"] .stButton > button {
    background-color: #c9a84c !important;
    color: #0a1628 !important;
    font-weight: 700 !important;
    border: none !important;
}
```

```
/* Chat input — fundo único escuro */
section[data-testid="stBottom"],
section[data-testid="stBottom"] > div,
div[data-testid="stChatInputContainer"],
div[data-testid="stChatInputContainer"] > div,
div[data-testid="stChatInputContainer"] textarea {
    background-color: #0f2133 !important;
    color: #ffffff !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
    box-shadow: none !important;
}

div[data-testid="stChatInputContainer"] textarea::placeholder {
    color: #94a3b8 !important;
}

/* Selectboxes do chat e globais */
div[data-testid="stSelectbox"] > div,
div[data-baseweb="select"] {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    color: #ffffff !important;
    border-radius: 6px !important;
}

/* Dropdown options */
ul[data-baseweb="menu"] {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
}

ul[data-baseweb="menu"] li {
    color: #ffffff !important;
}

ul[data-baseweb="menu"] li:hover {
    background-color: #1e3a5f !important;
}

/* Fundo geral e área principal */
.stApp, .main, .block-container {
    background-color: #0a1628 !important;
}

/* Sidebar fundo */
div[data-testid="stSidebar"] {
    background-color: #0f2133 !important;
}
```

```
/* Remover qualquer fundo branco residual */
div[data-testid="stAppViewContainer"] {
    background-color: #0a1628 !important;
}

Remova qualquer CSS anterior que entre em conflito com esses estilos.
Garanta que NÃO existe nenhum background branco ou cinza claro
em nenhum elemento do app.

Teste com: streamlit run app.py --server.headless true
```

precisa corrigir na pagina toda, em todas as abas e etc

18 de abr.

Manda isso para o Claude Code:

```
O app ainda tem elementos com fundo branco/claro em várias abas.
Faça uma correção global e definitiva de todo o CSS.

Substitua TODO o bloco de st.markdown CSS do app por este
bloco único e completo:

st.markdown("""
<style>

/* ═══════════════════════════════════════════════════════════
   BASE — fundo escuro em tudo
═══════════════════════════════════════════════════════════ */

.stApp,
.main,
.block-container,
div[data-testid="stAppViewContainer"],
div[data-testid="stMain"],
div[data-testid="stMainBlockContainer"] {
    background-color: #0a1628 !important;
    color: #ffffff !important;
}

div[data-testid="stSidebar"],
div[data-testid="stSidebar"] > div {
```

```
    background-color: #0f2133 !important;
}

section[data-testid="stBottom"] {
    background-color: #0a1628 !important;
}

header[data-testid="stHeader"] {
    background-color: #0a1628 !important;
}

.stDeployButton { display: none !important; }

/* ═══════════════════════════════════════════════════════════
   TIPOGRAFIA
═══════════════════════════════════════════════════════════ */
p, span, label, div, h1, h2, h3 {
    color: #ffffff !important;
}

/* ═══════════════════════════════════════════════════════════
   MÉTRICAS
═══════════════════════════════════════════════════════════ */
div[data-testid="stMetric"] {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
    padding: 16px !important;
}

div[data-testid="stMetricValue"] {
    color: #c9a84c !important;
    font-size: 28px !important;
    font-weight: 700 !important;
}

div[data-testid="stMetricLabel"] {
    color: #94a3b8 !important;
    font-size: 11px !important;
    text-transform: uppercase !important;
}

/* ═══════════════════════════════════════════════════════════
   ABAS
═══════════════════════════════════════════════════════════ */
div[data-testid="stTabs"] {
    background-color: #0a1628 !important;
}
```

```
button[data-baseweb="tab"] {
    background-color: #0a1628 !important;
    color: #94a3b8 !important;
}

button[data-baseweb="tab"][aria-selected="true"] {
    color: #c9a84c !important;
    border-bottom: 2px solid #c9a84c !important;
}

/* ═══════════════════════════════════════════════════════════
   BOTÕES
═══════════════════════════════════════════════════════════ */
.stButton > button {
    background-color: #0f2133 !important;
    color: #c9a84c !important;
    border: 1px solid #c9a84c !important;
    border-radius: 6px !important;
    font-weight: 600 !important;
}

.stButton > button:hover {
    background-color: #c9a84c !important;
    color: #0a1628 !important;
}

div[data-testid="stSidebar"] .stButton > button {
    background-color: #c9a84c !important;
    color: #0a1628 !important;
    font-weight: 700 !important;
    border: none !important;
    width: 100% !important;
}

/* ═══════════════════════════════════════════════════════════
   INPUTS E SELECTBOXES
═══════════════════════════════════════════════════════════ */
div[data-baseweb="select"],
div[data-baseweb="select"] > div,
div[data-baseweb="input"],
input, textarea {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    color: #ffffff !important;
    border-radius: 6px !important;
}

input::placeholder, textarea::placeholder {
    color: #94a3b8 !important;
}
```

```
ul[data-baseweb="menu"] {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
}

ul[data-baseweb="menu"] li {
    background-color: #0f2133 !important;
    color: #ffffff !important;
}

ul[data-baseweb="menu"] li:hover {
    background-color: #1e3a5f !important;
}

/* ═══════════════════════════════════════════════════════════════
   CHAT INPUT
═══════════════════════════════════════════════════════════════ */
div[data-testid="stChatInputContainer"],
div[data-testid="stChatInputContainer"] > div,
div[data-testid="stChatInputContainer"] textarea {
    background-color: #0f2133 !important;
    color: #ffffff !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
    box-shadow: none !important;
}

section[data-testid="stBottom"] > div {
    background-color: #0a1628 !important;
}

/* ═══════════════════════════════════════════════════════════════
   CHAT MENSAGENS
═══════════════════════════════════════════════════════════════ */
div[data-testid="stChatMessage"] {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
}

/* ═══════════════════════════════════════════════════════════════
   DATAFRAME / TABELA
═══════════════════════════════════════════════════════════════ */
div[data-testid="stDataFrame"],
div[data-testid="stDataFrame"] > div {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
```

```
}

/* ═══════════════════════════════════════════════════════════
   ALERTS — info, success, warning
═══════════════════════════════════════════════════════════ */
div[data-testid="stAlert"] {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
    color: #ffffff !important;
}

/* ═══════════════════════════════════════════════════════════
   SPINNER E OUTROS
═══════════════════════════════════════════════════════════ */
div[data-testid="stSpinner"] {
    color: #c9a84c !important;
}

div[data-testid="stCaption"] {
    color: #94a3b8 !important;
}

div[data-testid="stExpander"] {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
}

</style>
""", unsafe_allow_html=True)
```

Garanta que este é o ÚNICO bloco de CSS no app — remova
todos os outros st.markdown com CSS que existam no arquivo.

Depois atualize todos os gráficos plotly para usar este
template base em todos eles sem exceção:

```
G4_PLOTLY = dict(
    paper_bgcolor="#0a1628",
    plot_bgcolor="#0f2133",
    font=dict(color="#ffffff", family="sans-serif"),
    xaxis=dict(gridcolor="#1e3a5f", linecolor="#1e3a5f"),
    yaxis=dict(gridcolor="#1e3a5f", linecolor="#1e3a5f"),
)
```

E aplique em cada fig assim — sem passar margin no dicionário:
fig.update_layout(**G4_PLOTLY)

Teste com: streamlit run app.py --server.headless true

e confirme que TODAS as abas estão dark sem nenhum
elemento branco ou cinza claro visível.
Me mostra o resultado quando terminar.

?

acabei pedindo umas alterações ruins direto no claude code, preciso corrigir, quero que o app, não teenho opções de cor, quero que seja definitivo em dark mode, porém quero que ele volte ao que estava, que foi com seu prompt. mais ou menos dessa forma, mas no dark mode, esse print era do light mode, que quero eliminar do app inteiro

00:28

Entendido. O problema é que os botões de sugestão e alguns elementos ainda estão em light mode. Manda isso para o Claude Code:

O app está com elementos misturados entre dark e light mode.
Preciso que você:

1. Leia o arquivo app.py atual completo

2. Identifique e liste todos os elementos que ainda estão
   com fundo branco ou claro (especialmente os botões de
   sugestão do chat)

3. Substitua TODO o bloco CSS existente pelo seguinte
   bloco único e definitivo — delete qualquer outro
   st.markdown com CSS antes de adicionar este:

```
st.markdown("""
<style>

/* BASE */
.stApp, .main, .block-container,
div[data-testid="stAppViewContainer"],
div[data-testid="stMain"],
div[data-testid="stMainBlockContainer"] {
    background-color: #0a1628 !important;
    color: #ffffff !important;
}

div[data-testid="stSidebar"],
```

```
div[data-testid="stSidebar"] > div {
    background-color: #0f2133 !important;
}

section[data-testid="stBottom"],
section[data-testid="stBottom"] > div {
    background-color: #0a1628 !important;
}

header[data-testid="stHeader"] {
    background-color: #0a1628 !important;
}

.stDeployButton { display: none !important; }

/* TIPOGRAFIA */
p, span, label, div, h1, h2, h3, li, td, th {
    color: #ffffff !important;
}

/* MÉTRICAS */
div[data-testid="stMetric"] {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
    padding: 16px !important;
}
div[data-testid="stMetricValue"] {
    color: #c9a84c !important;
    font-size: 28px !important;
    font-weight: 700 !important;
}
div[data-testid="stMetricLabel"] {
    color: #94a3b8 !important;
    font-size: 11px !important;
    text-transform: uppercase !important;
}

/* ABAS */
div[data-testid="stTabs"],
div[data-testid="stTabContent"] {
    background-color: #0a1628 !important;
}
button[data-baseweb="tab"] {
    background-color: #0a1628 !important;
    color: #94a3b8 !important;
}
button[data-baseweb="tab"][aria-selected="true"] {
    color: #c9a84c !important;
    border-bottom: 2px solid #c9a84c !important;
```

```
    background-color: #0a1628 !important;
}

/* TODOS OS BOTÕES */
.stButton > button,
button[kind="secondary"],
button[kind="primary"] {
    background-color: #0f2133 !important;
    color: #c9a84c !important;
    border: 1px solid #c9a84c !important;
    border-radius: 6px !important;
    font-weight: 600 !important;
}
.stButton > button:hover {
    background-color: #c9a84c !important;
    color: #0a1628 !important;
}

/* BOTÃO ATUALIZAR SCORES NA SIDEBAR */
div[data-testid="stSidebar"] .stButton > button {
    background-color: #c9a84c !important;
    color: #0a1628 !important;
    font-weight: 700 !important;
    border: none !important;
    width: 100% !important;
}

/* SELECTBOXES E INPUTS */
div[data-baseweb="select"],
div[data-baseweb="select"] > div,
div[data-baseweb="input"],
input, textarea {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    color: #ffffff !important;
    border-radius: 6px !important;
}
input::placeholder, textarea::placeholder {
    color: #94a3b8 !important;
}

/* DROPDOWN OPTIONS */
ul[data-baseweb="menu"],
ul[data-baseweb="menu"] li {
    background-color: #0f2133 !important;
    color: #ffffff !important;
}
ul[data-baseweb="menu"] li:hover {
    background-color: #1e3a5f !important;
}
```

```
/* CHAT INPUT */
div[data-testid="stChatInputContainer"],
div[data-testid="stChatInputContainer"] > div,
div[data-testid="stChatInputContainer"] textarea {
    background-color: #0f2133 !important;
    color: #ffffff !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
    box-shadow: none !important;
}

/* CHAT MENSAGENS */
div[data-testid="stChatMessage"],
div[data-testid="stChatMessage"] > div {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
    color: #ffffff !important;
}

/* DATAFRAME */
div[data-testid="stDataFrame"],
div[data-testid="stDataFrame"] > div,
div[data-testid="stDataFrame"] iframe {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
}

/* ALERTS */
div[data-testid="stAlert"],
div[data-testid="stAlert"] > div {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
    color: #ffffff !important;
}

/* EXPANDER */
div[data-testid="stExpander"] {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
}

/* CAPTION */
div[data-testid="stCaption"],
.stCaption {
    color: #94a3b8 !important;
```

```
}

/* SPINNER */
div[data-testid="stSpinner"] > div {
    border-top-color: #c9a84c !important;
}

/* MULTISELECT */
span[data-baseweb="tag"] {
    background-color: #1e3a5f !important;
    color: #ffffff !important;
}

/* PROGRESS BAR */
div[data-testid="stProgress"] > div > div {
    background-color: #c9a84c !important;
}

</style>
""", unsafe_allow_html=True)
```

4. Para os botões de sugestão do chat especificamente,
   NÃO use st.button() — substitua por st.markdown com
   links clicáveis estilizados assim:

```
sugestoes = [
    "Quais meus 3 melhores deals para fechar essa semana?",
    "Quais setores têm mais chance de fechar?",
    "Por que meu score médio está baixo?",
    "Quais deals devo abandonar?"
]

cols = st.columns(2)
for i, s in enumerate(sugestoes):
    if cols[i % 2].button(s, key=f"sug_{i}", use_container_width=True):
        st.session_state.chat_input_value = s
```

   Isso garante que os botões herdam o CSS global corretamente.

5. Atualize todos os gráficos plotly com este template
   — aplique em TODOS sem exceção:

```
G4_PLOTLY = dict(
    paper_bgcolor="#0a1628",
    plot_bgcolor="#0f2133",
    font=dict(color="#ffffff", family="sans-serif"),
    xaxis=dict(gridcolor="#1e3a5f", linecolor="#1e3a5f"),
    yaxis=dict(gridcolor="#1e3a5f", linecolor="#1e3a5f"),
)
fig.update_layout(**G4_PLOTLY)
```

Teste com: streamlit run app.py --server.headless true
Confirme que TODAS as abas estão 100% dark.
Me manda print de todas as abas quando terminar.

o menu lateral esta cinza, quero que fique com um tom de azul, e alguns titulos, cards com mensagens de avisos, inputs com a cor dourado, padrao g4 educação

00:34

Manda isso para o Claude Code:

Ajuste fino de cores no CSS global do app.py seguindo
a identidade visual G4 Educação:

1. SIDEBAR — tom azul escuro ao invés de cinza:
```
div[data-testid="stSidebar"],
div[data-testid="stSidebar"] > div {
   background-color: #051525 !important;
   border-right: 1px solid #1e3a5f !important;
}
```

2. LABELS E TÍTULOS DA SIDEBAR — dourado:
```
div[data-testid="stSidebar"] label,
div[data-testid="stSidebar"] p,
div[data-testid="stSidebar"] span {
   color: #c9a84c !important;
   font-size: 11px !important;
   text-transform: uppercase !important;
   font-weight: 600 !important;
   letter-spacing: 0.5px !important;
}
```

3. TÍTULOS DAS SEÇÕES NO CONTEÚDO PRINCIPAL — dourado:
```
h1, h2, h3,
div[data-testid="stMarkdown"] h1,
div[data-testid="stMarkdown"] h2,
div[data-testid="stMarkdown"] h3 {
   color: #c9a84c !important;
   font-weight: 700 !important;
}
```

4. CARDS DE ALERTA — borda e ícone dourado:

```
div[data-testid="stAlert"] {
    background-color: #0f2133 !important;
    border: 1px solid #c9a84c !important;
    border-left: 4px solid #c9a84c !important;
    border-radius: 8px !important;
}
div[data-testid="stAlert"] p,
div[data-testid="stAlert"] span {
    color: #ffffff !important;
}
```

5. INPUTS E SELECTBOXES — borda dourada no foco:

```
div[data-baseweb="select"]:focus-within,
div[data-baseweb="input"]:focus-within,
textarea:focus,
input:focus {
    border: 1px solid #c9a84c !important;
    box-shadow: 0 0 0 1px #c9a84c !important;
}
```

6. PLACEHOLDER DOS INPUTS — tom dourado suave:

```
input::placeholder,
textarea::placeholder {
    color: #8a7040 !important;
}
```

7. MULTISELECT TAGS — fundo azul com texto dourado:

```
span[data-baseweb="tag"] {
    background-color: #1e3a5f !important;
    color: #c9a84c !important;
    border: 1px solid #c9a84c !important;
}
```

8. CHAT INPUT — borda dourada:

```
div[data-testid="stChatInputContainer"],
div[data-testid="stChatInputContainer"] > div,
div[data-testid="stChatInputContainer"] textarea {
    background-color: #0f2133 !important;
    border: 1px solid #c9a84c !important;
    border-radius: 8px !important;
    color: #ffffff !important;
    box-shadow: none !important;
}
```

9. PROGRESS BAR — dourada:

```
div[data-testid="stProgress"] > div > div {
    background-color: #c9a84c !important;
}
```

```
div[data-testid="stProgress"] > div {
   background-color: #1e3a5f !important;
   border-radius: 4px !important;
}

10. SEPARADORES — azul escuro:
hr {
   border-color: #1e3a5f !important;
}

Adicione esses estilos ao bloco CSS existente sem
remover nenhum dos estilos anteriores.

Teste com: streamlit run app.py --server.headless true
```

Me manda prints de todas as abas quando terminar.

não ficou legal ainda. o atual esta no primeiro print, quero ele mais parecido com o segundo print

00:37

Entendido. O segundo print tem um visual muito mais limpo e profissional — sidebar mais escura, labels em uppercase dourado, métricas com valores grandes em dourado, tabela com fundo escuro e texto claro.

Manda isso para o Claude Code:

```
Quero que o visual do app fique exatamente como estava
antes das últimas alterações de CSS.

Faça o seguinte:

1. Use git para ver o histórico de commits:
   git log --oneline -10

2. Identifique o commit antes das alterações de CSS
   que quebraram o visual

3. Recupere apenas o arquivo app.py desse commit:
```

```
   git show HASH_DO_COMMIT:submissions/guilherme-pardin/solution/app.py >
app_backup.py

4. Me mostre o hash do commit encontrado e confirme
   que o app_backup.py foi criado

Não substitua o app.py ainda — quero confirmar
o commit certo antes.
```

Me mostra o output do git log.

queria deixar o front end com o desing do g4, podemos pegar como referencia a pagina de vagas deles, lá da pra ter uma referencia do design e tudo mais18 de abr.Boa ideia — personalizar o visual para o estilo do G4 mostra atenção ao detalhe e deixa a entrega mais profissional. Antes de montar o prom

pasted

quero neste padrao de volta:

00:41

Manda isso para o Claude Code:

```
Leia o arquivo app.py atual e substitua TODO o bloco de CSS
por este abaixo — é o CSS definitivo e correto do projeto.
Delete qualquer outro st.markdown com CSS antes de adicionar.

st.markdown("""
<style>

/* BASE */
.stApp, .main, .block-container,
div[data-testid="stAppViewContainer"],
div[data-testid="stMain"],
div[data-testid="stMainBlockContainer"] {
    background-color: #0a1628 !important;
    color: #ffffff !important;
}

div[data-testid="stSidebar"],
```

```
div[data-testid="stSidebar"] > div {
    background-color: #0f2133 !important;
    border-right: 1px solid #1e3a5f !important;
}

section[data-testid="stBottom"],
section[data-testid="stBottom"] > div {
    background-color: #0a1628 !important;
}

header[data-testid="stHeader"] {
    background-color: #0a1628 !important;
}

.stDeployButton { display: none !important; }

/* TIPOGRAFIA GERAL */
p, span, div, h1, h2, h3, li, td, th {
    color: #ffffff !important;
}

/* LABELS SIDEBAR — uppercase dourado */
div[data-testid="stSidebar"] label {
    color: #c9a84c !important;
    font-size: 11px !important;
    text-transform: uppercase !important;
    font-weight: 600 !important;
    letter-spacing: 0.5px !important;
}

/* MÉTRICAS */
div[data-testid="stMetric"] {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
    padding: 16px !important;
}
div[data-testid="stMetricValue"] {
    color: #c9a84c !important;
    font-size: 28px !important;
    font-weight: 700 !important;
}
div[data-testid="stMetricLabel"] {
    color: #94a3b8 !important;
    font-size: 11px !important;
    text-transform: uppercase !important;
}

/* ABAS */
div[data-testid="stTabs"],
```

```
div[data-testid="stTabContent"] {
    background-color: #0a1628 !important;
}
button[data-baseweb="tab"] {
    background-color: #0a1628 !important;
    color: #94a3b8 !important;
}
button[data-baseweb="tab"][aria-selected="true"] {
    color: #c9a84c !important;
    border-bottom: 2px solid #c9a84c !important;
    background-color: #0a1628 !important;
}

/* BOTÕES GERAIS */
.stButton > button {
    background-color: #0f2133 !important;
    color: #c9a84c !important;
    border: 1px solid #c9a84c !important;
    border-radius: 6px !important;
    font-weight: 600 !important;
}
.stButton > button:hover {
    background-color: #c9a84c !important;
    color: #0a1628 !important;
}

/* BOTÃO ATUALIZAR SCORES */
div[data-testid="stSidebar"] .stButton > button {
    background-color: #c9a84c !important;
    color: #0a1628 !important;
    font-weight: 700 !important;
    border: none !important;
    width: 100% !important;
}

/* SELECTBOXES */
div[data-baseweb="select"],
div[data-baseweb="select"] > div {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    color: #ffffff !important;
    border-radius: 6px !important;
}
div[data-baseweb="select"]:focus-within {
    border: 1px solid #c9a84c !important;
}

/* DROPDOWN OPTIONS */
ul[data-baseweb="menu"],
ul[data-baseweb="menu"] li {
```

```
    background-color: #0f2133 !important;
    color: #ffffff !important;
}
ul[data-baseweb="menu"] li:hover {
    background-color: #1e3a5f !important;
}

/* INPUTS */
input, textarea {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    color: #ffffff !important;
    border-radius: 6px !important;
}
input::placeholder, textarea::placeholder {
    color: #94a3b8 !important;
}

/* CHAT INPUT */
div[data-testid="stChatInputContainer"],
div[data-testid="stChatInputContainer"] > div,
div[data-testid="stChatInputContainer"] textarea {
    background-color: #0f2133 !important;
    color: #ffffff !important;
    border: 1px solid #c9a84c !important;
    border-radius: 8px !important;
    box-shadow: none !important;
}

/* CHAT MENSAGENS */
div[data-testid="stChatMessage"],
div[data-testid="stChatMessage"] > div {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
}

/* DATAFRAME */
div[data-testid="stDataFrame"],
div[data-testid="stDataFrame"] > div {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
}

/* ALERTS */
div[data-testid="stAlert"],
div[data-testid="stAlert"] > div {
    background-color: #0f2133 !important;
    border-left: 4px solid #c9a84c !important;
```

```
    border-radius: 8px !important;
}

/* MULTISELECT TAGS */
span[data-baseweb="tag"] {
    background-color: #1e3a5f !important;
    color: #c9a84c !important;
    border: 1px solid #c9a84c !important;
}

/* PROGRESS BAR */
div[data-testid="stProgress"] > div {
    background-color: #1e3a5f !important;
    border-radius: 4px !important;
}
div[data-testid="stProgress"] > div > div {
    background-color: #c9a84c !important;
}

/* CAPTION */
div[data-testid="stCaption"], .stCaption {
    color: #94a3b8 !important;
}

/* SEPARADORES */
hr { border-color: #1e3a5f !important; }

/* EXPANDER */
div[data-testid="stExpander"] {
    background-color: #0f2133 !important;
    border: 1px solid #1e3a5f !important;
    border-radius: 8px !important;
}

</style>
""", unsafe_allow_html=True)
```

Depois garanta que:

1. O header usa este HTML exato:

```
st.markdown("""
<div style="margin-bottom:24px;">
  <span style="color:#c9a84c;font-weight:800;font-size:28px;">G4</span><span
style="color:#ffffff;font-weight:400;font-size:28px;">Lead Scorer</span>
  <div style="color:#94a3b8;font-size:13px;margin-top:4px;">Priorização inteligente de
pipeline</div>
</div>
""", unsafe_allow_html=True)
```

2. A sidebar usa este HTML exato no topo:

```
st.sidebar.markdown("""
<div style="margin-bottom:24px;padding-bottom:16px;border-bottom:1px solid #1e3a5f;">
  <span style="color:#c9a84c;font-weight:800;font-size:20px;">G4</span><span
style="color:#ffffff;font-weight:400;font-size:20px;">Lead Scorer</span>
</div>
""", unsafe_allow_html=True)

3. Todos os gráficos plotly usam este template —
   sem margin no dicionário principal:

G4_PLOTLY = dict(
    paper_bgcolor="#0a1628",
    plot_bgcolor="#0f2133",
    font=dict(color="#ffffff", family="sans-serif"),
    xaxis=dict(gridcolor="#1e3a5f", linecolor="#1e3a5f"),
    yaxis=dict(gridcolor="#1e3a5f", linecolor="#1e3a5f"),
)
fig.update_layout(**G4_PLOTLY)
fig.update_layout(margin=dict(l=20, r=20, t=40, b=20))

4. Barras dos gráficos:
   Tier A: #4ade80
   Tier B: #c9a84c
   Tier C: #f87171

Teste com: streamlit run app.py --server.headless true
Confirme que todas as abas estão 100% dark.
```
Me manda prints das 4 abas quando terminar.

otimize o prompt, para ele não ser cortado na hora de copiar no claude code

00:42

```
Leia o app.py atual. Faça 4 coisas:

1. DELETE todo CSS existente e substitua por este bloco único:

st.markdown("""<style>
.stApp,.main,.block-container,div[data-testid="stAppViewContainer"],div[data-
testid="stMain"],div[data-testid="stMainBlockContainer"]{background-color:#0a1628!
important;color:#fff!important}
```

```
div[data-testid="stSidebar"],div[data-testid="stSidebar"]>div{background-color:#0f2133!
important;border-right:1px solid #1e3a5f!important}
section[data-testid="stBottom"],section[data-testid="stBottom"]>div{background-
color:#0a1628!important}
header[data-testid="stHeader"]{background-color:#0a1628!important}
.stDeployButton{display:none!important}
p,span,div,h1,h2,h3,li,td,th{color:#fff!important}
div[data-testid="stSidebar"] label{color:#c9a84c!important;font-size:11px!important;text-
transform:uppercase!important;font-weight:600!important;letter-spacing:.5px!important}
div[data-testid="stMetric"]{background-color:#0f2133!important;border:1px solid #1e3a5f!
important;border-radius:8px!important;padding:16px!important}
div[data-testid="stMetricValue"]{color:#c9a84c!important;font-size:28px!important;font-
weight:700!important}
div[data-testid="stMetricLabel"]{color:#94a3b8!important;font-size:11px!important;text-
transform:uppercase!important}
div[data-testid="stTabs"],div[data-testid="stTabContent"]{background-color:#0a1628!
important}
button[data-baseweb="tab"]{background-color:#0a1628!important;color:#94a3b8!important}
button[data-baseweb="tab"][aria-selected="true"]{color:#c9a84c!important;border-
bottom:2px solid #c9a84c!important;background-color:#0a1628!important}
.stButton>button{background-color:#0f2133!important;color:#c9a84c!important;border:1px
solid #c9a84c!important;border-radius:6px!important;font-weight:600!important}
.stButton>button:hover{background-color:#c9a84c!important;color:#0a1628!important}
div[data-testid="stSidebar"] .stButton>button{background-color:#c9a84c!
important;color:#0a1628!important;font-weight:700!important;border:none!
important;width:100%!important}
div[data-baseweb="select"],div[data-baseweb="select"]>div{background-color:#0f2133!
important;border:1px solid #1e3a5f!important;color:#fff!important;border-radius:6px!
important}
div[data-baseweb="select"]:focus-within{border:1px solid #c9a84c!important}
ul[data-baseweb="menu"],ul[data-baseweb="menu"] li{background-color:#0f2133!
important;color:#fff!important}
ul[data-baseweb="menu"] li:hover{background-color:#1e3a5f!important}
input,textarea{background-color:#0f2133!important;border:1px solid #1e3a5f!
important;color:#fff!important;border-radius:6px!important}
input::placeholder,textarea::placeholder{color:#94a3b8!important}
div[data-testid="stChatInputContainer"],div[data-
testid="stChatInputContainer"]>div,div[data-testid="stChatInputContainer"]
textarea{background-color:#0f2133!important;color:#fff!important;border:1px solid
#c9a84c!important;border-radius:8px!important;box-shadow:none!important}
div[data-testid="stChatMessage"],div[data-testid="stChatMessage"]>div{background-
color:#0f2133!important;border:1px solid #1e3a5f!important;border-radius:8px!important}
div[data-testid="stDataFrame"],div[data-testid="stDataFrame"]>div{background-
color:#0f2133!important;border:1px solid #1e3a5f!important;border-radius:8px!important}
div[data-testid="stAlert"],div[data-testid="stAlert"]>div{background-color:#0f2133!
important;border-left:4px solid #c9a84c!important;border-radius:8px!important}
span[data-baseweb="tag"]{background-color:#1e3a5f!important;color:#c9a84c!
important;border:1px solid #c9a84c!important}
div[data-testid="stProgress"]>div{background-color:#1e3a5f!important;border-radius:4px!
important}
```

```
div[data-testid="stProgress"]>div>div{background-color:#c9a84c!important}
div[data-testid="stCaption"],.stCaption{color:#94a3b8!important}
hr{border-color:#1e3a5f!important}
div[data-testid="stExpander"]{background-color:#0f2133!important;border:1px solid
#1e3a5f!important;border-radius:8px!important}
</style>""", unsafe_allow_html=True)
```

2. Header principal — substitua o título por:

```
st.markdown('<div style="margin-bottom:24px"><span style="color:#c9a84c;font-
weight:800;font-size:28px;">G4</span><span style="color:#fff;font-weight:400;font-
size:28px;">Lead Scorer</span><div style="color:#94a3b8;font-size:13px;margin-
top:4px;">Priorização inteligente de pipeline</div></div>', unsafe_allow_html=True)
```

3. Topo da sidebar — adicione antes dos filtros:

```
st.sidebar.markdown('<div style="margin-bottom:24px;padding-bottom:16px;border-
bottom:1px solid #1e3a5f;"><span style="color:#c9a84c;font-weight:800;font-
size:20px;">G4</span><span style="color:#fff;font-weight:400;font-size:20px;">Lead
Scorer</span></div>', unsafe_allow_html=True)
```

4. Todos os gráficos plotly — aplique em TODOS sem exceção:

```
G4_PLOTLY = dict(paper_bgcolor="#0a1628", plot_bgcolor="#0f2133",
font=dict(color="#ffffff", family="sans-serif"), xaxis=dict(gridcolor="#1e3a5f",
linecolor="#1e3a5f"), yaxis=dict(gridcolor="#1e3a5f", linecolor="#1e3a5f"))

fig.update_layout(**G4_PLOTLY)
fig.update_layout(margin=dict(l=20, r=20, t=40, b=20))
```

Cores das barras: Tier A=#4ade80, Tier B=#c9a84c, Tier C=#f87171

Teste: streamlit run app.py --server.headless true
Confirme que todas as abas estão 100% dark.

?
?
?
o front end não esta legal. quero deixar ele com o padrão da G4 educação, com as cores da empresa e o padrão de design das suas aplicações.

00:49

Agora tenho referências muito melhores — os prints do site principal da G4 mostram o design system real deles com muito mais detalhe.

Design system real extraído dos prints:

- Fundo: `#0a1628` (azul marinho escuro)
- Navbar/cards: `#0d1f35` (azul ligeiramente mais claro)
- Dourado principal: `#c9a060` (mais quente que o anterior)
- Dourado botões: `#b8943f` (tom terroso dourado)
- Texto primário: `#ffffff`
- Texto secundário: `#8a9bb0` (azul acinzentado)
- Bordas: `#1a3a5c`
- Títulos: fonte sans-serif bold, uppercase em alguns casos
- Botões: fundo dourado `#c9a060` com texto escuro `#0a1628`, border-radius suave

Manda isso para o Claude Code:

Leia o app.py atual. Substitua TODO o CSS existente e todos os elementos visuais pelo design system real da G4 Educação extraído do site g4educacao.com.

1. DELETE todo st.markdown com CSS existente.

2. Adicione este bloco CSS único no início do app:

```
st.markdown("""<style>
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&display=swap');

*{font-family:'Inter',sans-serif!important}

.stApp,.main,.block-container,div[data-testid="stAppViewContainer"],div[data-testid="stMain"],div[data-testid="stMainBlockContainer"]{background-color:#0a1628!important;color:#fff!important}

div[data-testid="stSidebar"],div[data-testid="stSidebar"]>div{background-color:#0d1f35!important;border-right:1px solid #1a3a5c!important}

section[data-testid="stBottom"],section[data-testid="stBottom"]>div{background-color:#0a1628!important}

header[data-testid="stHeader"]{background-color:#0a1628!important}

.stDeployButton{display:none!important}

p,span,div,h1,h2,h3,li,td,th{color:#fff!important}

div[data-testid="stSidebar"] label{color:#c9a060!important;font-size:10px!important;text-transform:uppercase!important;font-weight:700!important;letter-spacing:1px!important}
```

div[data-testid="stMetric"]{background-color:#0d1f35!important;border:1px solid #1a3a5c!important;border-radius:12px!important;padding:20px!important}
div[data-testid="stMetricValue"]{color:#c9a060!important;font-size:32px!important;font-weight:800!important}
div[data-testid="stMetricLabel"]{color:#8a9bb0!important;font-size:10px!important;text-transform:uppercase!important;letter-spacing:1px!important;font-weight:600!important}

div[data-testid="stTabs"],div[data-testid="stTabContent"]{background-color:#0a1628!important}
button[data-baseweb="tab"]{background-color:#0a1628!important;color:#8a9bb0!important;font-weight:500!important;font-size:14px!important}
button[data-baseweb="tab"][aria-selected="true"]{color:#c9a060!important;border-bottom:2px solid #c9a060!important;background-color:#0a1628!important;font-weight:700!important}

.stButton>button{background-color:#0d1f35!important;color:#c9a060!important;border:1px solid #c9a060!important;border-radius:8px!important;font-weight:600!important;font-size:14px!important;padding:8px 16px!important}
.stButton>button:hover{background-color:#c9a060!important;color:#0a1628!important}

div[data-testid="stSidebar"] .stButton>button{background-color:#c9a060!important;color:#0a1628!important;font-weight:700!important;border:none!important;width:100%!important;border-radius:8px!important}

div[data-baseweb="select"],div[data-baseweb="select"]>div{background-color:#0d1f35!important;border:1px solid #1a3a5c!important;color:#fff!important;border-radius:8px!important}
div[data-baseweb="select"]:focus-within{border:1px solid #c9a060!important}

ul[data-baseweb="menu"],ul[data-baseweb="menu"] li{background-color:#0d1f35!important;color:#fff!important}
ul[data-baseweb="menu"] li:hover{background-color:#1a3a5c!important}

input,textarea{background-color:#0d1f35!important;border:1px solid #1a3a5c!important;color:#fff!important;border-radius:8px!important}
input::placeholder,textarea::placeholder{color:#8a9bb0!important}

div[data-testid="stChatInputContainer"],div[data-testid="stChatInputContainer"]>div,div[data-testid="stChatInputContainer"] textarea{background-color:#0d1f35!important;color:#fff!important;border:1px solid #c9a060!important;border-radius:10px!important;box-shadow:none!important}

div[data-testid="stChatMessage"],div[data-testid="stChatMessage"]>div{background-color:#0d1f35!important;border:1px solid #1a3a5c!important;border-radius:10px!important}

div[data-testid="stDataFrame"],div[data-testid="stDataFrame"]>div{background-color:#0d1f35!important;border:1px solid #1a3a5c!important;border-radius:10px!important}

div[data-testid="stAlert"],div[data-testid="stAlert"]>div{background-color:#0d1f35!

important;border-left:4px solid #c9a060!important;border-radius:8px!important}

span[data-baseweb="tag"]{background-color:#1a3a5c!important;color:#c9a060!important;border:1px solid #c9a060!important;border-radius:4px!important}

div[data-testid="stProgress"]>div{background-color:#1a3a5c!important;border-radius:4px!important}
div[data-testid="stProgress"]>div>div{background-color:#c9a060!important}

div[data-testid="stCaption"],.stCaption{color:#8a9bb0!important}
hr{border-color:#1a3a5c!important}
div[data-testid="stExpander"]{background-color:#0d1f35!important;border:1px solid #1a3a5c!important;border-radius:10px!important}
</style>""", unsafe_allow_html=True)

3. Header principal — substitua por:

st.markdown(' G4Lead Scorer Priorização inteligente de pipeline ', unsafe_allow_html=True)

4. Topo da sidebar:

st.sidebar.markdown(' G4Lead Scorer ', unsafe_allow_html=True)

5. Todos os gráficos plotly — aplique em TODOS:

G4_PLOTLY = dict(paper_bgcolor="#0a1628", plot_bgcolor="#0d1f35",
font=dict(color="#ffffff",family="Inter,sans-serif"),
xaxis=dict(gridcolor="#1a3a5c",linecolor="#1a3a5c"),
yaxis=dict(gridcolor="#1a3a5c",linecolor="#1a3a5c"))
fig.update_layout(**G4_PLOTLY)
fig.update_layout(margin=dict(l=20,r=20,t=40,b=20))

Cores barras: Tier A=#4ade80, Tier B=#c9a060, Tier C=#f87171

Teste: streamlit run app.py --server.headless true
Confirme todas as abas 100% dark sem elemento branco.
Me manda prints das 4 abas quando terminar.

o design ta cada vez pior

00:53

Entendo a frustração. O problema é que o Streamlit tem limitações sérias de CSS — muitos elementos não respondem bem a customização via `st.markdown`.

A melhor solução agora é dar um passo atrás e recuperar o visual que estava funcionando — o do segundo print que você mandou antes, que estava limpo e legível.

Manda isso para o Claude Code:

Execute: git log --oneline -15

Me mostre o output completo.

Com o histórico de commits eu identifico exatamente o ponto onde o visual estava bom e recupero só o app.py daquele momento, sem perder nenhuma funcionalidade que adicionamos depois.

quero voltar a cor do app original que esta no submission, mas sem tirar as funções que foram adicionadas

10:47

Manda isso para o Claude Code:

Execute os comandos abaixo e me mostre o output:

git log --oneline -20

9bd86fa Add chat exports to process log 12364d9 Add screenshots to process log d49c566 Submission: Guilherme Pardin — Challenge 003 Lead Scorer 4aed364 feat: PR-only

submissions — remove email option bcdfd2e docs: add baseline warning — AI-only submissions won't pass 4b55509 refactor: clean up repo structure — remove duplicates, add missing files d4c8fc7 feat: add README.md for each challenge with detailed descriptions d91427f feat: AI Master Challenge — initial structure with 4 challenges

10:48

O commit `d49c566` é o original com o visual funcionando. Vamos recuperar só o `app.py` e `scoring.py` desse commit e reapliar as funções novas por cima.

Manda isso para o Claude Code:

Execute estes comandos:

```
git show d49c566:submissions/guilherme-pardin/solution/app.py > app_original.py
git show d49c566:submissions/guilherme-pardin/solution/scoring.py > scoring_original.py
```

Depois me mostre as diferenças entre o app_original.py e o app.py atual:
```
diff app_original.py app.py
```

Quero ver o que foi adicionado depois do commit original.
Me mostra o output do diff.

Executado 5 comandos Output completo. Aqui está o resumo do que foi adicionado ao app original: Adicionado nesta sessão (322 linhas novas): CategoriaO que mudouImports`compute_win_rates`, `MONTH_NAMES_PT`, `chat_completion`CSSBloco dark mode completo (inexistente no original)Plotly`G4_PLOTLY` + `G4_MARGIN` (inexistentes no original)Feature caption`days_in_pipeline` → `Sazonalidade`Cache function`get_scored_df()` → `get_all_data()` retornando também `month_wr`SidebarSeletor de modelo Claude (Sonnet/Haiku) adicionadoHeader`st.markdown("# 📊 Lead Scorer")` → `st.title()` + `st.caption()`Abas3 abas → 4 abas (adicionada aba "💬 Chat com IA")Tab 1CSS da tabela removido do inline e movido pro bloco globalTab 3 (Chat)Aba inteira nova — filtros inline, sugestões, `st.chat_input`, `build_pipeline_context()`Tab 4 (Gestor)Gráfico de win rate por mês adicionado (era Tab 3 no original) Removido do original: CSS inline da tabela pipeline (`.ls-wrap` com `background:#1e1e1e`),

`TIER_COLORS` como dict de cores bootstrap.

Mostrar mais
10:49

Perfeito. Agora está claro o que precisa ser feito — restaurar o visual original e manter só as funções novas. Manda isso para o Claude Code:

```
Faça o seguinte:

1. Copie o app_original.py para app.py:
   cp app_original.py app.py

2. Agora adicione de volta APENAS as funcionalidades novas,
   sem alterar nada do visual/CSS original:

IMPORT — adicione no topo junto aos outros imports:
from scoring import compute_win_rates, MONTH_NAMES_PT
from ai_insights import get_recommendation, chat_completion

SIDEBAR — após os filtros existentes, adicione o seletor de modelo:
st.sidebar.markdown("---")
model_options = {"claude-sonnet-4": "claude-sonnet-4-20250514", "claude-haiku-4":
"claude-haiku-4-5-20251001"}
model_label = st.sidebar.selectbox("Modelo Claude", list(model_options.keys()))
model = model_options[model_label]

CACHE — substitua get_scored_df() por get_all_data() que retorna
também month_wr calculado via compute_win_rates()

ABAS — mude de 3 para 4 abas adicionando "Chat com IA" entre
"Análise IA" e "Gestor". O conteúdo da aba Chat é:

  - 3 colunas no topo: selectbox Vendedor, Tier, Região
    (filtros independentes da sidebar — afetam só o contexto do chat)
  - st.caption mostrando contexto atual: "X deals | Vendedor Y"
  - Botões de sugestão (só quando histórico vazio):
    "Quais meus 3 melhores deals para fechar essa semana?"
    "Quais setores têm mais chance de fechar?"
    "Por que meu score médio está baixo?"
    "Quais deals devo abandonar?"
  - st.chat_message para histórico
  - st.chat_input("Pergunte sobre seu pipeline...")
  - Botão "Limpar conversa" reseta st.session_state
```

- Sem API key: st.warning pedindo a chave
- Contexto para IA: top 50 deals por score com campos
  account, deal_stage, score, tier, sector, sales_agent,
  effective_value, sazonalidade e win rates
- Passa model para chat_completion()

ABA GESTOR — adicione após os gráficos existentes o gráfico
de win rate por mês:
- Barras verticais, mês em português no eixo X
- Win rate % no eixo Y
- Verde se >=65%, dourado se >=55%, vermelho se <55%
- Texto acima de cada barra: "73% (247 deals)"
- Caption explicando que meses com win rate alto geram
  mais pontos de sazonalidade no score

CAPTION ABA 1 — atualize para mencionar Sazonalidade
no lugar de days_in_pipeline:
"Sazonalidade: win rate histórico do mês de entrada (20pts)"

3. Após editar teste com:
   streamlit run app.py --server.headless true

4. Confirme que o visual está idêntico ao original
   e todas as 4 abas funcionam.

Me mostra o resultado quando terminar.

queria corrigir esse grafico, deixar ele mais legivel, parece esta mto comprimido, n em espaço para ele ter o tabanho normal.

10:55

Manda isso para o Claude Code:

Corrija o gráfico de barras horizontal "Score por feature"
na Aba 2 Análise IA.

Problemas atuais:
- Gráfico muito comprimido horizontalmente
- Labels cortados (ex: "20 pl" ao invés de "20 pts")
- Barras muito finas

Correções:

1. Force o gráfico a ocupar largura total da coluna:
   use_container_width=True no st.plotly_chart()

2. Aumente a altura do gráfico para 320px:
   fig.update_layout(height=320)

3. Aumente o tamanho das barras (espessura):
   fig.update_traces(width=0.6)

4. Labels fora da barra com espaço suficiente:
   textposition="outside"
   fig.update_layout(xaxis=dict(range=[0, 38]))

5. Aumente a margem direita para os labels não cortarem:
   margin=dict(l=160, r=60, t=30, b=40)

6. Fonte dos labels do eixo Y maior e legível:
   fig.update_layout(yaxis=dict(tickfont=dict(size=13)))

Teste com: streamlit run app.py --server.headless true

quero mexer no front agora, acha melhor darmos um commit antes de subirmos o front?

11:03

Sim, definitivamente. Faz o commit agora para salvar tudo que foi adicionado — sazonalidade, chat, seletor de modelo, gráfico de win rate — antes de mexer no frontend.

Se o frontend quebrar de novo, você tem um ponto seguro para voltar.

No terminal do VSCode:

bash
```
cd "/Users/sacefilmes/Desktop/challenge G4/lead-scorer/ai-master-challenge"
git add -f submissions/guilherme-pardin/
git commit -m "Add chat, seasonality feature, model selector and win rate chart"
git push origin submission/guilherme-pardin
```
Me mostra o output quando terminar.

You are a senior UI/UX designer and front-end engineer specialized in high-ticket B2B landing pages. Your task is to design and implement a front-end interface inspired by the visual identity of G4 Educação (Brazil), following a premium, corporate, high-conversion style. DO NOT create a generic design. Follow the system strictly:

========== 🎨 COLOR SYSTEM ==========

Use this exact palette: - Primary Background (Deep Blue): `#072B3A` - Secondary Background: `#0F3F52` - Accent (Gold): `#C89B5A` - Accent Hover: `#D4A96A` - Text Primary: `#FFFFFF` - Text Secondary: `#A0A0A0` - Dark Elements: `#0B0B0B` Rules: - Background must be dark (deep blue dominant) - Gold must ONLY be used for CTA and highlights - Maintain high contrast at all times ========== 🧱 DESIGN STYLE ========== - Premium, minimal, corporate - High-ticket product feeling - Clean spacing (lots of whitespace) - No visual clutter - Strong visual hierarchy ========== 📐 LAYOUT STRUCTURE ========== Follow this structure: 1. HERO SECTION - Left: Headline + subheadline + CTA - Right: Image of people (authority/business style) - Strong contrast and focus on conversion 2. TRUST BAR - Metrics / social proof (numbers, icons) 3. CONTENT SECTIONS - Alternating blocks (text + image) - Clean separation using spacing (not heavy borders) 4. CTA SECTION - Strong gold button - Clear action text ========== 🔤 TYPOGRAPHY ========== - Use modern sans-serif (Inter or similar) - Large bold headlines - Medium body text - Clear hierarchy: H1 > H2 > Body > Caption ========== 🔘 BUTTONS ========== Primary CTA: - Background: Gold (`#C89B5A`) - Text: Dark (`#0B0B0B`) - Rounded corners - Hover: lighter gold (`#D4A96A`) - Slight scale or brightness effect on hover ========== 🎯 UX PRINCIPLES ========== - Everything should guide user toward the CTA - Clear reading flow (top → bottom, left → right) - Avoid distractions - Optimize for conversion ========== ⚙️ OUTPUT ========== Generate: - Clean, production-ready front-end code - Use React + Tailwind (preferred) - Component-based structure - Responsive design (mobile + desktop) ========== 🚫 DO NOT: ========== - Do not use random colors - Do not use gradients unless very subtle - Do not create a “startup playful” design - Do not overuse animations - Do not break the premium aesthetic ========== 🎯 GOAL ========== The final result should look like a high-end executive education landing page, similar to G4 Educação, focused on authority, clarity, and conversion.

Mostrar mais
11:05

Esse prompt é para uma landing page — não é o que precisamos agora. O que você quer é aplicar o design system G4 no Streamlit app existente.

Antes de mexer no frontend, me responde:

P: O que você quer fazer com o frontend agora? R: Melhorar o visual do Streamlit app existente

11:05

Certo. O problema que tivemos antes é que o Streamlit tem limitações sérias de CSS — muitos elementos ignoram estilos customizados.

A abordagem mais eficiente agora é diferente das anteriores: em vez de tentar estilizar cada elemento via CSS seletor, vamos usar o sistema de temas nativo do Streamlit que é mais estável e previsível.

Manda isso para o Claude Code:

```
Vamos aplicar o design G4 do jeito correto no Streamlit —
usando o sistema de temas nativo + CSS cirúrgico mínimo.

1. Crie o arquivo .streamlit/config.toml na pasta solution/:

[theme]
base = "dark"
primaryColor = "#C89B5A"
backgroundColor = "#072B3A"
secondaryBackgroundColor = "#0F3F52"
textColor = "#FFFFFF"
font = "sans serif"

2. No app.py, mantenha APENAS este CSS mínimo no st.markdown,
   removendo todo o resto:

st.markdown("""<style>
.stDeployButton{display:none!important}
header[data-testid="stHeader"]{background-color:#072B3A!important}
div[data-testid="stMetricValue"]{color:#C89B5A!important;font-size:28px!important;font-
```

```
weight:800!important}
div[data-testid="stMetricLabel"]{color:#A0A0A0!important;font-size:10px!important;text-transform:uppercase!important;letter-spacing:1px!important}
button[data-baseweb="tab"][aria-selected="true"]{color:#C89B5A!important;border-bottom:2px solid #C89B5A!important}
div[data-testid="stSidebar"] .stButton>button{background-color:#C89B5A!important;color:#0B0B0B!important;font-weight:700!important;border:none!important;width:100%!important;border-radius:8px!important}
.stButton>button{border:1px solid #C89B5A!important;color:#C89B5A!important;border-radius:8px!important}
.stButton>button:hover{background-color:#C89B5A!important;color:#0B0B0B!important}
span[data-baseweb="tag"]{background-color:#0F3F52!important;color:#C89B5A!important;border:1px solid #C89B5A!important}
div[data-testid="stChatInputContainer"] textarea{border:1px solid #C89B5A!important;border-radius:8px!important}
div[data-testid="stAlert"]{border-left:4px solid #C89B5A!important}
hr{border-color:#1a3a5c!important}
</style>""", unsafe_allow_html=True)
```

3. Mantenha o header e sidebar exatamente assim:

Header:

```
st.markdown('<div style="margin-bottom:32px;padding-bottom:20px;border-bottom:1px solid #1a3a5c"><span style="color:#C89B5A;font-weight:800;font-size:30px;">G4</span><span style="color:#fff;font-weight:300;font-size:30px;">Lead Scorer</span><div style="color:#A0A0A0;font-size:12px;margin-top:6px;text-transform:uppercase;letter-spacing:1.5px;">Priorização inteligente de pipeline</div></div>', unsafe_allow_html=True)
```

Sidebar:

```
st.sidebar.markdown('<div style="margin-bottom:24px;padding-bottom:14px;border-bottom:1px solid #1a3a5c;"><span style="color:#C89B5A;font-weight:800;font-size:18px;">G4</span><span style="color:#fff;font-weight:300;font-size:18px;">Lead Scorer</span></div>', unsafe_allow_html=True)
```

4. Atualize todos os gráficos plotly com:

```
G4_PLOTLY = dict(
    paper_bgcolor="#072B3A",
    plot_bgcolor="#0F3F52",
    font=dict(color="#FFFFFF", family="sans-serif"),
    xaxis=dict(gridcolor="#1a3a5c", linecolor="#1a3a5c"),
    yaxis=dict(gridcolor="#1a3a5c", linecolor="#1a3a5c")
)
fig.update_layout(**G4_PLOTLY)
fig.update_layout(margin=dict(l=20, r=20, t=40, b=20))
```

Cores barras: Tier A=#4ade80, Tier B=#C89B5A, Tier C=#f87171

5. Teste com: streamlit run app.py --server.headless true
   Confirme que o config.toml foi criado e o app carregou

com o novo tema.

Me manda print quando terminar — o tema nativo vai resolver a maioria dos problemas de cor de fundo automaticamente.

ficou bem legal agora, consigo aumentar alguns elementos? quero aumentar o tamanho do titulo G4 Lead scorer no menu lateral esquerdo, e alguns titulos da aba analise de IA

11:11

Ficou muito melhor — o tema nativo resolveu. Manda isso para o Claude Code:

Dois ajustes de tipografia no app.py:

1. SIDEBAR — aumente o título G4 Lead Scorer:
Substitua o st.sidebar.markdown do título por:

st.sidebar.markdown(' G4Lead Scorer ', unsafe_allow_html=True)

2. ABA ANÁLISE IA — aumente os títulos das seções:

Título "Score por feature":
st.markdown(' Score por feature ', unsafe_allow_html=True)

Título "Detalhes do deal":
st.markdown(' Detalhes do deal ', unsafe_allow_html=True)

Título "Análise com IA":
st.markdown(' Análise com IA ', unsafe_allow_html=True)

Teste com: streamlit run app.py --server.headless true